Project-1 MindTrack

STEP 1: Clone & Run React App Locally

```
git clone https://github.com/Vennilavan12/Brain-Tasks-App.git
cd Brain-Tasks-App
npm install
npm start
```

App runs on:

```
http://localhost:3000
```

STEP 2: Create Dockerfile (Dockerize App)

Create `Dockerfile` in root directory:

```
# Build Stage
FROM node:18 AS build
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
RUN npm run build

# Production Stage
FROM nginx:alpine
COPY --from=build /app/build /usr/share/nginx/html
EXPOSE 3000
CMD ["nginx", "-g", "daemon off;"]
```

STEP 3: Build & Test Docker Image

```
docker build -t brain-tasks-app .
docker run -d -p 3000:80 brain-tasks-app
```

check

```
http://localhost:3000
```

STEP 4: Create AWS ECR Repository

Login to AWS

```
aws configure
```

Create ECR Repo

```
aws ecr create-repository \
--repository-name brain-tasks-app \
--region ap-south-1
```

Copy Repository URI:

```
123456789.dkr.ecr.ap-south-1.amazonaws.com/brain-tasks-app
```

Login to ECR

```
aws ecr get-login-password --region ap-south-1 | \
docker login --username AWS --password-stdin <ECR_URI>
```

Push Image

```
docker tag brain-tasks-app:latest <ECR_URI>:latest

docker push <ECR_URI>:latest
```

STEP 5: Create EKS Cluster

Install eksctl

```
curl -LO
https://github.com/weaveworks/eksctl/releases/latest/download/eksctl_Linux_
amd64.tar.gz
tar -xzf eksctl_Linux_amd64.tar.gz
sudo mv eksctl /usr/local/bin
```

Create Cluster

```
eksctl create cluster \
--name brain-cluster \
--region ap-south-1 \
--nodegroup-name nodes \
--node-type t3.micro \
--nodes 2
```

Verify

```
kubectl get nodes
```

STEP 6: Kubernetes Deployment & Service

Create folder:

```
mkdir k8s
cd k8s
```

---

deployment.yml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: brain-app
spec:
  replicas: 2
  selector:
    matchLabels:
      app: brain
  template:
    metadata:
      labels:
        app: brain
    spec:
      containers:
      - name: brain-container
        image: <ECR_URI>:latest
        ports:
        - containerPort: 80
```

---

service.yml

```
apiVersion: v1
kind: Service
metadata:
  name: brain-service
spec:
  type: LoadBalancer
  selector:
    app: brain
  ports:
    - port: 3000
      targetPort: 80
```

Deploy

```
kubectl apply -f deployment.yml
kubectl apply -f service.yml
```

Get Public IP

```
kubectl get svc
```

Access from browser using EXTERNAL-IP.

STEP 7: CodeBuild Configuration

Create `buildspec.yml` in repo root:

```
version: 0.2
```

```
env:
  variables:
    AWS_DEFAULT_REGION: ap-south-1
    ECR_REPO: brain-tasks-app

phases:
  pre_build:
    commands:
      - aws ecr get-login-password --region $AWS_DEFAULT_REGION |
        docker login --username AWS --password-stdin
$AWS_ACCOUNT_ID.dkr.ecr.$AWS_DEFAULT_REGION.amazonaws.com

  build:
    commands:
      - docker build -t $ECR_REPO .
      - docker tag $ECR_REPO:latest
$AWS_ACCOUNT_ID.dkr.ecr.$AWS_DEFAULT_REGION.amazonaws.com/$ECR_REPO:latest

  post_build:
    commands:
      - docker push
$AWS_ACCOUNT_ID.dkr.ecr.$AWS_DEFAULT_REGION.amazonaws.com/$ECR_REPO:latest
```

Create CodeBuild Project

AWS Console → CodeBuild → Create Project

Setting	Value
Source	GitHub
Environment	Ubuntu Standard
Privileged	Enabled
Buildspec	buildspec.yml

STEP 8: CodeDeploy Setup (For EKS)

Create `appspec.yml`:

```
version: 0.0
Resources:
  - TargetService:
      Type: AWS::EKS::Service
      Properties:
        TaskDefinition: brain-app
        LoadBalancerInfo:
          ContainerName: brain-container
          ContainerPort: 80
```

For EKS, CodeDeploy is mainly used with blue/green deployment via ALB.

CodeDeploy Application

AWS Console → CodeDeploy → Create Application

Type: `EKS`

Deployment Type: Blue/Green

STEP 9: Push to GitHub Using CLI

```
git init
git add .
git commit -m "Initial DevOps deployment setup"
git remote add origin https://github.com/username/brain-tasks-app.git
git push -u origin main
```

STEP 10: Create CodePipeline

AWS Console → CodePipeline → Create Pipeline

Stage 1: Source

- Provider: GitHub
- Repo: Brain-Tasks-App
- Branch: main

Stage 2: Build

- Provider: CodeBuild
- Project: brain-build

Stage 3: Deploy

Option 1: CodeDeploy
Option 2: Lambda Script to run kubectl
Option 3: Custom Script

```
Build → Push → kubectl apply
```

Example Deploy Script (deploy.sh)

```
aws eks update-kubeconfig --name brain-cluster

kubectl apply -f k8s/deployment.yml
kubectl apply -f k8s/service.yml
```

Add this in CodeBuild post_build if using custom deploy.

STEP 11: Monitoring (CloudWatch Logs)

Enable Logs

CodeBuild → Project → Logs

✓ Enable CloudWatch Logs

View Logs

AWS Console → CloudWatch → Log Groups

Check:

```
/aws/codebuild/brain-build
/aws/codepipeline/brain-pipeline
```

FINAL ARCHITECTURE

```
GitHub
   ↓
CodePipeline
   ↓
CodeBuild → Docker Build → ECR
   ↓
Deploy Script / CodeDeploy
   ↓
EKS (Kubernetes)
   ↓
LoadBalancer → Users
```